

Software-Defined Networking with OpenFlow

440-2216/01 Quality of Service • Lecture 09

Jan Rozhon • Department of Telecommunications, FEECS

Where we are going

Part 1

Why SDN

Limits of box-by-box control,
the three planes, the road to
SDN

↔ Question 2

Part 2

Architecture

Planes, northbound /
southbound / east-west
interfaces, SDN vs NFV

↔ Steps 1, 3

Part 3

OpenFlow

Flow tables, pipeline, groups,
meters, ports, channel,
messages

↔ Step 5, Questions 2–3

Part 4

OpenFlow vs VXLAN-EVPN

Centralized flow
programming vs a distributed
BGP overlay

background

One question runs through the lecture: **where is the forwarding decision made, and what does that placement cost in scale, reliability, and delay?**

What you should be able to do

1. Distinguish the data, control, and management planes and explain what SDN changes about their placement.
2. Describe the SDN architecture and the role of the northbound, southbound, and east-west interfaces; distinguish SDN from NFV.
3. Read an OpenFlow flow entry, trace a packet through a multi-table pipeline, and explain groups, meters, and reserved ports.
4. Explain the OpenFlow message types and the reactive flow set-up, and estimate its effect on the first packet of a flow.
5. Compare OpenFlow with VXLAN-EVPN in control-plane placement, scale, failure behaviour, and QoS handling.

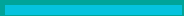
Keep asking: **which plane does this component belong to, and what happens to the traffic if it fails?**

SDN separates deciding where a packet goes from forwarding it

and puts the deciding into software that sees the whole network

Part 1

Why software-defined networking



Three planes in every network device

Data plane

Forwards, drops, queues, or rewrites each packet according to local tables.

Works per packet, at line rate:

nanoseconds to microseconds.

Control plane

Computes the tables: routing protocols (OSPF, BGP), spanning tree, MAC

learning. Reacts to events:

milliseconds to seconds.

Management plane

Configures devices and collects telemetry: CLI, SNMP, NETCONF.

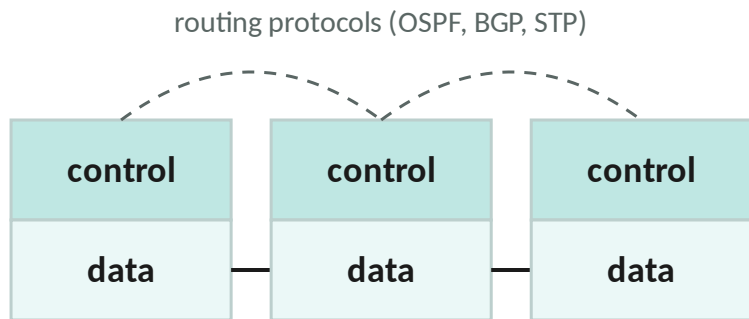
Operator-driven: **minutes to days.**

In a traditional router or switch all three planes share one chassis, one vendor, and one operating system.

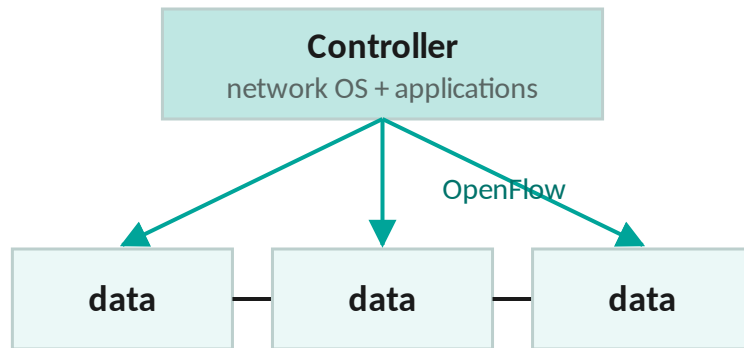
Kurose & Ross call the resulting data-plane operation *match plus action*: look up header fields, apply an action. SDN generalizes which fields may be matched and which program fills the table.

Traditional networks: control distributed in every box

TRADITIONAL — DISTRIBUTED CONTROL



SDN — LOGICALLY CENTRALIZED CONTROL



Traditional: every box decides for itself and the network state is the sum of many local views.

SDN: switches only forward; one program with a global view decides for all of them.

In a traditional network, the forwarding state is the **emergent result** of many independent control planes running distributed protocols. In SDN, a controller with a **global view** computes it and installs it.

Limitations of traditional networks

Limited scaling of operations

Hundreds of devices configured one by one, each with its own CLI; policy is scattered over many configuration files.

Vendor lock-in

Control software is bundled with the hardware; interoperability rests on whatever protocols the vendors chose to implement.

Slow innovation

A new protocol needs standardization, then vendor implementation, then a hardware refresh — years (compare IPv6 with a mobile app release cycle).

Complicated resource migration

A virtual machine or container that moves between hosts takes its addresses and policies with it; the network must follow within seconds.

Limited resource sharing

Isolation by VLANs only: 4094 IDs, and no isolation of CPU, tables, or bandwidth between tenants or between production and experiments.

Hard to verify and debug

No component knows the complete forwarding state, so questions such as "can host A reach host B?" have no single place to be answered.

The idea of SDN

Separate the control plane from the data plane:

- network elements become simple packet forwarders,
- one logically centralized program — the **controller** — computes the forwarding logic for all elements,
- the two are connected by an open, standardized interface.

SDN applies the same step to packet networks: decisions are made off the forwarding path, by software that can be replaced without replacing the switch.

An old idea in telephony

Common Channel Signalling (SS7) moved call control out of the voice channel onto a separate signalling network, with central intelligence in the Intelligent Network (IN).

The road to SDN

Year	Milestone	Contribution
1990s	Active networks	programmable packet processing; no deployment path
2003	IETF ForCES	control/forwarding split in a router; new hardware needed
2004	RCP	central BGP route selection; limited to existing protocols
2005	4D project	clean-slate decision / dissemination / discovery / data
2007	Ethane (Stanford)	central access-policy controller: the direct ancestor
2008	OpenFlow, NOX	open interface to existing switch tables; first network OS
2013	Google B4	SDN-controlled inter-data-centre WAN in production
2014	P4	programmable parser and match-action pipeline

Part 2

SDN architecture and interfaces

Architecture — planes and interfaces

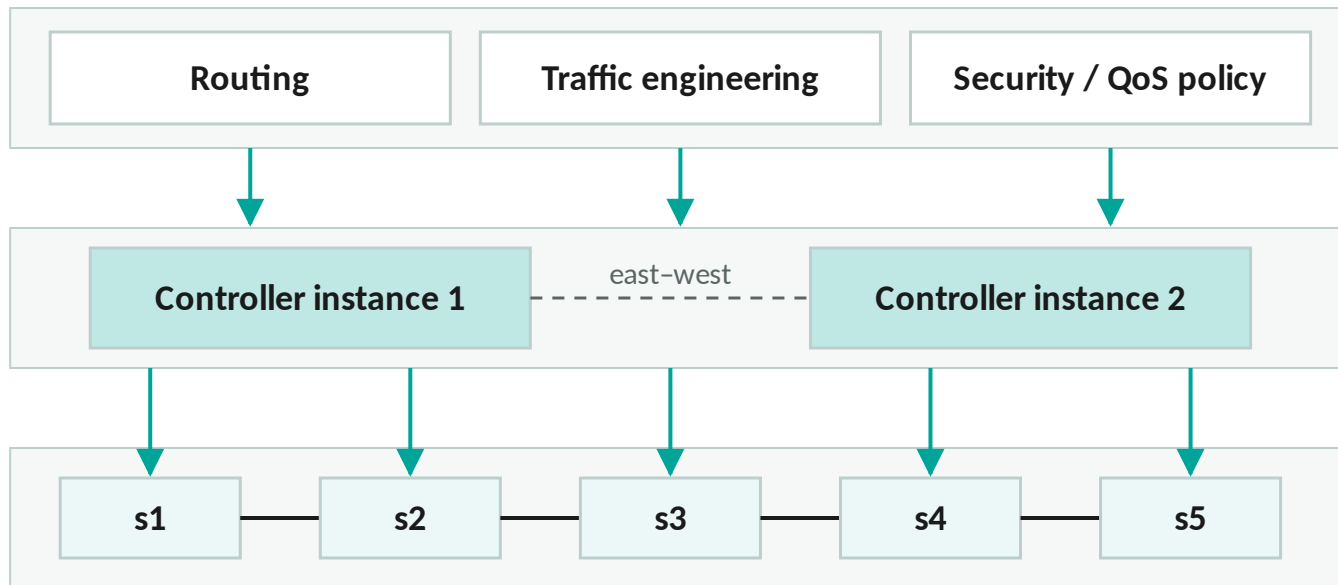
APPLICATION PLANE

northbound API
REST, gRPC

CONTROL PLANE

southbound API
OpenFlow, P4Runtime

DATA PLANE



Only the southbound side has a widely adopted standard; each controller defines its own northbound API.

SDN interfaces

Northbound interface (NBI)

Between applications and the controller. Exposes topology, flows, and intents.

Not standardized; commonly REST or gRPC, specific to each controller (ONOS, OpenDaylight, Ryu).

Southbound interface (SBI)

Between the controller and network elements; the ONF calls it the *Control-Data-Plane Interface* (CDPI).

Standardized: OpenFlow is the classic example; also P4Runtime, NETCONF/YANG, OVSDB, gNMI.

East-west interface

Between controllers, to share state across domains or replicas.

Proposed as **SDNi** (IETF draft); in practice each controller cluster uses its own protocol (Raft, distributed stores).

The southbound interface is where interoperability matters most: it is what lets one controller program switches from different vendors.

The controller — a network operating system

The controller maintains a **global network view** — topology, hosts, flows, statistics — and offers it to applications, much as an operating system abstracts hardware for programs.

- Topology discovery (LLDP via Packet-Out/Packet-In)
- Host tracking, path computation
- Flow-rule installation and consistency
- Statistics collection

Controller	Origin	Language
NOX / POX	Nicira, Stanford	C++ / Python
Ryu	NTT	Python
Floodlight	Big Switch	Java
OpenDaylight	Linux Foundation	Java
ONOS	ON.Lab / ONF	Java

Challenges of a centralized controller

Scalability

One controller must process every Packet-In and hold the whole network state.

Mitigation: eliminate redundant route computation, index affected switches for fast updates, partition the network (e.g. an IGP between partitions), proactive rules.

Reliability

The controller is a potential single point of failure.

Mitigation: hot-standby and clustered controllers (OpenFlow master/slave roles), and switch fail modes that keep forwarding.

Consistency

Replicas must agree on the network state, and each needs its own information feed.

Mitigation: distributed stores (Onix, ONOS); consistent-update schemes so that no packet sees a mix of old and new rules.

"Logically centralized, physically distributed" is the answer of every production controller — at the price of the consistency problems of any distributed system.

Advantages of SDN

Virtualization and resource sharing — slices isolated down to CPU and table level; production and experiments can coexist on one network.

No vendor lock-in — commodity (white-box) switches, control software chosen separately.

Rapid and continuous development — the control logic evolves after the switch is bought, at software speed.

Dynamic resource allocation — a change in demand is reflected in forwarding within one control loop.

Complete overview of the network — easier debugging, verification, and monitoring from one place.

Lower costs — simpler hardware and centralized orchestration.

Each advantage is a consequence of the same step: the decision logic is software with a global view, not firmware in each box.

SDN vs NFV — complementary, not competing

Software-Defined Networking

Separates control from forwarding. Concerns **where packets go**: layers 2–4 of switches and routers.

Standards: ONF, IETF.

Network Function Virtualization

Replaces monolithic middleboxes (firewalls, load balancers, IDS, NAT) with **software on commodity servers**, composed from modules.

Standards: ETSI NFV ISG (2012).

Example of modular NFV: the Snort IDS = IP defragmenter + preprocessing + misuse-detection engine + logger; each module can be scaled or replaced on its own.

Together: SDN steers traffic through a **service chain** of virtual functions that NFV instantiates on demand.

Checkpoint — which interface?

An operator writes an application that moves video traffic off a congested link. Which interfaces does its decision cross before the first switch changes its behaviour, and which of them is standardized?

Predict first · discuss with a neighbour · then reveal

Two. The application asks the controller through the **northbound** API (REST/gRPC, controller-specific, not standardized). The controller translates the request into flow entries and installs them through the **southbound** interface (e.g. OpenFlow Flow-Mod, standardized). If the controller is a cluster, the new state is also replicated east-west.

Part 3

OpenFlow



OpenFlow

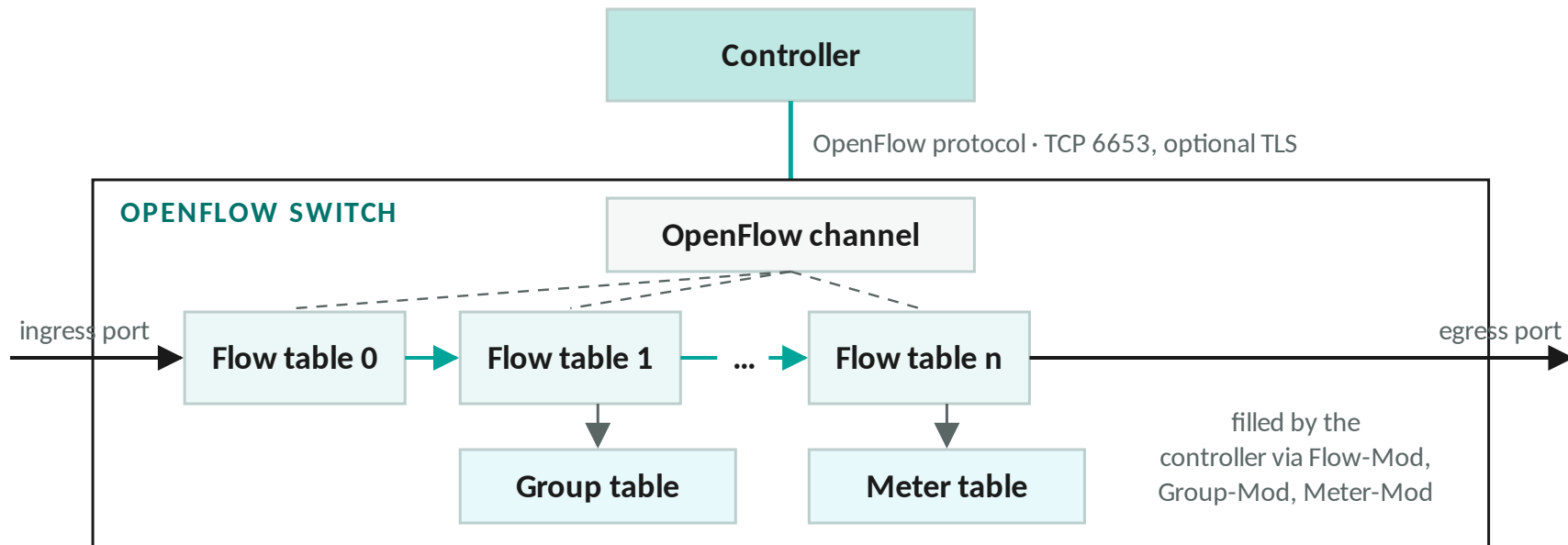
A protocol between a network element and an external controller for installing forwarding rules into the element's **flow tables**; it implements the southbound interface (CDPI).

- Proposed at Stanford in 2008 (McKeown et al.); specified by the ONF since 2011
- **No custom hardware needed:** flow tables (TCAM) were already in 2008 switches; the vendor only opens an interface to them
- The switch stays closed — only the communication interface is open

Version	Year	Key addition
1.0	2009	single table, 12 match fields
1.1	2011	multiple tables, groups
1.2	2011	extensible matches (OXM), IPv6
1.3	2012	meters, per-flow QoS; long-term support
1.4	2013	bundles, flow monitoring
1.5.1	2015	egress tables — last release

No new version since 2015; the ONF shifted to P4 and P4Runtime. OpenFlow **remains in active use**, above all in Open vSwitch.

The OpenFlow switch



The switch hardware stays closed; only the tables and the channel that fills them are standardized.

Every OpenFlow switch has at least one **flow table**; the controller adds, modifies, and deletes entries in it. Optional **group** and **meter** tables extend what an entry can do.

A flow entry

ONE FLOW ENTRY — OPENFLOW 1.3

Match fields	Priority	Counters	Instructions	Timeouts	Cookie	Flags
in_port=1, eth_type=0x0800, ip_dst=10.0.0.2, ip_dscp=46	100 highest wins	packets, bytes, duration	meter:1, apply: output:2	idle 10 s, hard 0 (none)	opaque tag of controller	notify on removal

Match fields and priority identify the entry; a packet is handled by the highest-priority entry it matches.

The table-miss entry wildcards every field at priority 0: send to controller, drop, or go to a later table.

Match fields — ingress port, Ethernet, VLAN, IPv4/IPv6, DSCP, TCP/UDP ports, tunnel ID, metadata, ... any field may be wildcarded or masked.

Timeouts — `idle_timeout` removes an unused entry;
`hard_timeout` removes it regardless. Expiry can trigger a Flow-Removed message.

Worked example — which entry matches?

Table 0 of a switch:

Prio	Match	Instructions
300	ip, ip_dscp=46	set_queue:1, output:2
200	tcp, tp_dst=22	output:CONTROLLER
100	in_port=1	output:2
0	(any)	drop

1. A VoIP RTP packet from port 1, DSCP EF?

2. An SSH packet from port 1, DSCP 0?

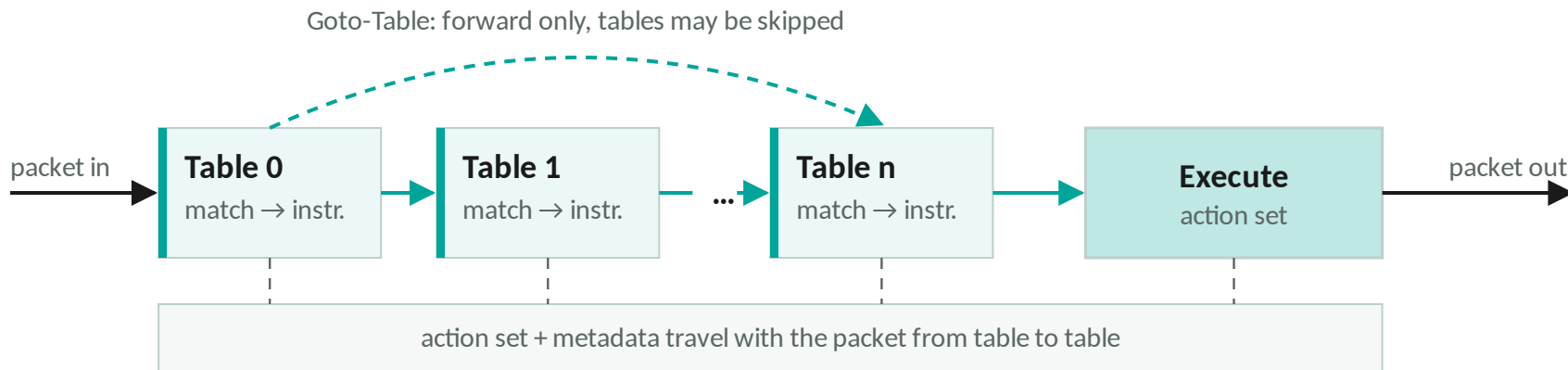
3. An ARP request from port 3?

1. **Prio 300** — queue 1, port 2. It also matches prio 100, but the higher priority wins.

2. **Prio 200** — sent to the controller, not forwarded.

3. **Prio 0** — the table-miss entry drops it: ARP is not IP and did not arrive on port 1.

Pipeline processing



Apply-Actions modifies the packet immediately; Write-Actions and Clear-Actions edit the action set, which is executed once, when an entry without Goto-Table ends the pipeline.

Flow tables are chained: an entry's **Goto-Table** instruction continues the lookup in a later table, which allows cascaded, modular processing (e.g. table 0 = access control, table 1 = QoS marking, table 2 = forwarding). If no entry matches, the **table-miss** entry decides: next table, controller, or drop.

Group table and meter table

Group table — an extra level of indirection, referenced by `group: <id>` from a flow entry. A group holds a list of **action buckets**:

Type	Behaviour
<code>all</code>	every bucket — multicast, flooding
<code>select</code>	one bucket (hash) — load sharing, ECMP
<code>indirect</code>	single bucket — shared next hop
<code>fast failover</code>	first live bucket — local protection

Meter table (since OpenFlow 1.3) — per-flow rate measurement and limiting, referenced by the `meter` instruction. An entry contains:

- meter identifier,
- **meter bands**: rate, burst, and type — `drop` or `dscp_remark`,
- counters.

A meter band is Lecture 03's token-bucket **policer**: excess traffic is dropped or re-marked to a lower DSCP class.

Ports

Physical ports — the switch's hardware interfaces.

Logical ports — defined outside OpenFlow: link aggregation groups, tunnels, loopbacks. A packet from a logical port can carry a **Tunnel-ID** as metadata.

Reserved ports — used as the target of an output action for generic forwarding.

Reserved port	Meaning
ALL	flood to all ports except ingress
CONTROLLER	encapsulate and send to the controller
TABLE	submit to the pipeline (Packet-Out only)
IN_PORT	send back out through the ingress port
LOCAL	the switch's own network stack
NORMAL	traditional non-OpenFlow forwarding
FLOOD	traditional flooding, respecting STP

The OpenFlow channel

- Carries configuration requests from the controller and events from the switch
- **TCP**, optionally **TLS**; IANA port **6653** (6633 in older implementations)
- **Connection initiated by the switch**
- Usually a separate management network (**out-of-band**); in-band control is possible
- Several controllers: roles *equal*, *master*, *slave* (1.2+)

When the channel fails

fail-secure — keep installed entries, drop packets that would go to the controller

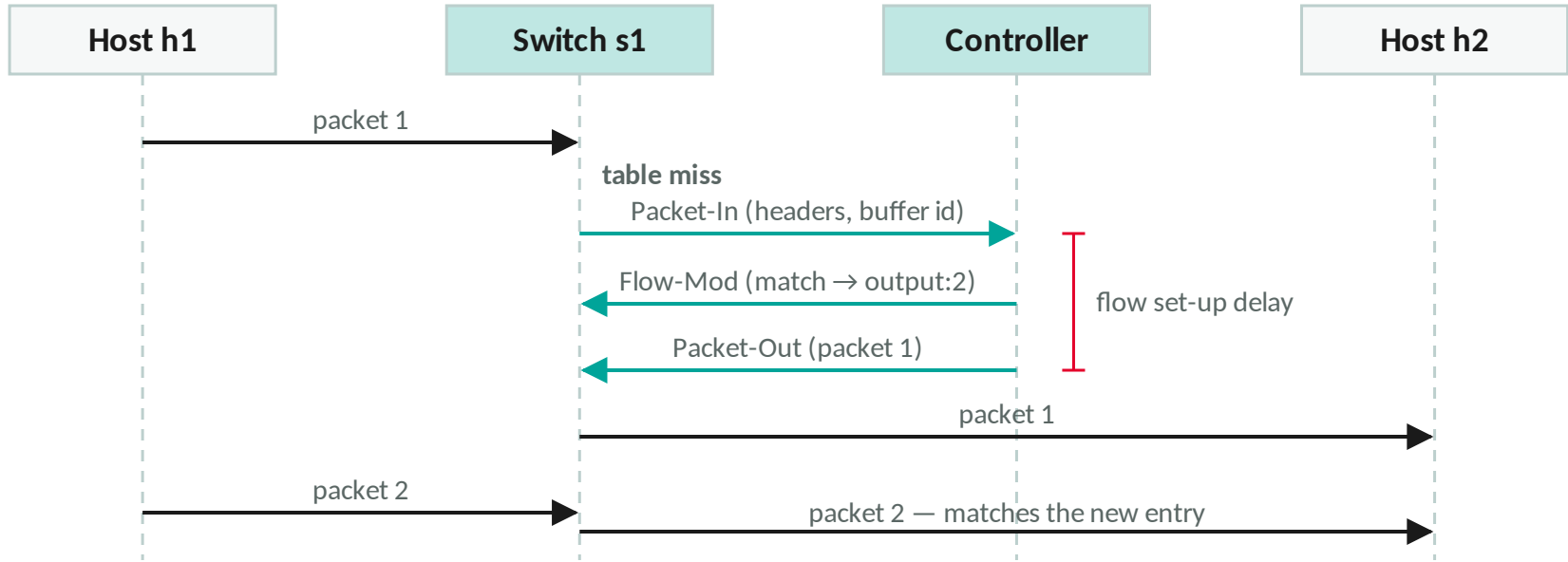
fail-standalone — behave as an ordinary learning switch (NORMAL)

OpenFlow message types

Type	Initiated by	Purpose	Examples
Controller-to-switch	controller	configure, modify, and read the switch	Features-Request/Reply, Flow-Mod, Group-Mod, Meter-Mod, Port-Mod, Packet-Out, Multipart (statistics), Barrier, Role-Request
Asynchronous	switch, on an event	report without being asked	Packet-In, Flow-Removed, Port-Status
Symmetric	either side, unsolicited	keep the session alive	Hello, Echo-Request/Reply, Error, Experimenter

Every message starts with the same 8-byte header: version, type, length, transaction ID (`xid`). A Barrier-Reply tells the controller that all earlier requests have been processed — the only way to know that a rule is really installed.

Reactive flow set-up



Only the first packet of a flow pays for the controller round trip; the rest stay in the data plane.

Reactive vs proactive flow installation

Reactive

Entries are installed on demand, after a Packet-In.

- + small tables, only active flows
- + fine-grained, per-flow policy
- – first packet waits for the controller
- – controller load grows with the flow arrival rate
- – traffic for new flows stops if the controller is lost

Proactive

Entries are installed in advance, from topology and policy.

- + no set-up delay, no per-flow controller load
- + forwarding survives a controller outage
- – tables must hold every possible rule (TCAM size)
- – coarser policy, usually wildcard rules

Production networks are mostly proactive, with reactive handling only for exceptions — the same trade-off the controller's scalability challenge forces.

Checkpoint — the slow first ping

In Mininet with a reactive learning controller, the first ping between two hosts takes 5–10 ms. every following one about 0.05 ms. Explain the difference. What would change after del-flows on the switch?

Predict first · discuss with a neighbour · then reveal

The first echo request (and the ARP exchange before it) **misses** in the empty flow table: the switch sends a Packet-In, the controller computes a decision and answers with Flow-Mod and Packet-Out, and the reply does the same in the other direction. Later packets match the installed entries and stay in the data plane. After del-flows the next ping pays the **flow set-up delay again**, and the controller reinstalls the entries.

Checkpoint — the controller crashes

A switch in **fail-secure** mode loses its only controller. What happens to an ongoing video call, and what happens to a call that starts a minute later?

Predict first · discuss with a neighbour · then reveal

The ongoing call continues — its entries are already in the flow table and the data plane does not need the controller (unless an idle or hard timeout expires them). **The new call fails** if its packets need a Packet-In: in fail-secure mode those packets are dropped. With proactive rules both calls would work; in **fail-standalone** mode the switch would fall back to ordinary learning and forward both, without the controller's policy.

OpenFlow as a QoS tool

Classification — match on any header field, including `ip_dscp`, ports, and addresses (Lecture 03's classifier, programmable per flow).

Marking — `set_field:ip_dscp` re-marks at the network edge.

Policing — meter bands drop or re-mark traffic above a rate.

Scheduling — `set_queue` selects a port queue; the queue's rate and discipline are configured outside OpenFlow.

Measurement — per-flow, per-port, and per-queue counters polled by Multipart requests give the controller a live traffic matrix.

Traffic engineering — the controller places flows on paths according to the measured load.

Google B4 (2013)

SDN traffic engineering drove inter-data-centre WAN links close to 100 % utilization, against 30–40 % typical for conventionally routed WANs.

Beyond OpenFlow

The limits of a fixed protocol

- Every new header field needs a new specification: from 12 fields (1.0) to more than 40 (1.4)
- The switch chip still decides which fields it can parse
- Hardware table sizes (TCAM) limit wildcard entries

P4 and P4Runtime

- **P4** describes the parser and the match-action pipeline itself; the program is compiled onto a programmable chip
- **P4Runtime** is the southbound API that fills the tables of that program — the role OpenFlow plays for a fixed pipeline

SDN as an architecture outlived OpenFlow as a protocol: controllers such as ONOS now speak OpenFlow, P4Runtime, NETCONF, and gNMI side by side.

Part 4

OpenFlow vs VXLAN-EVPN

The data-centre problem

A multi-tenant data centre needs:

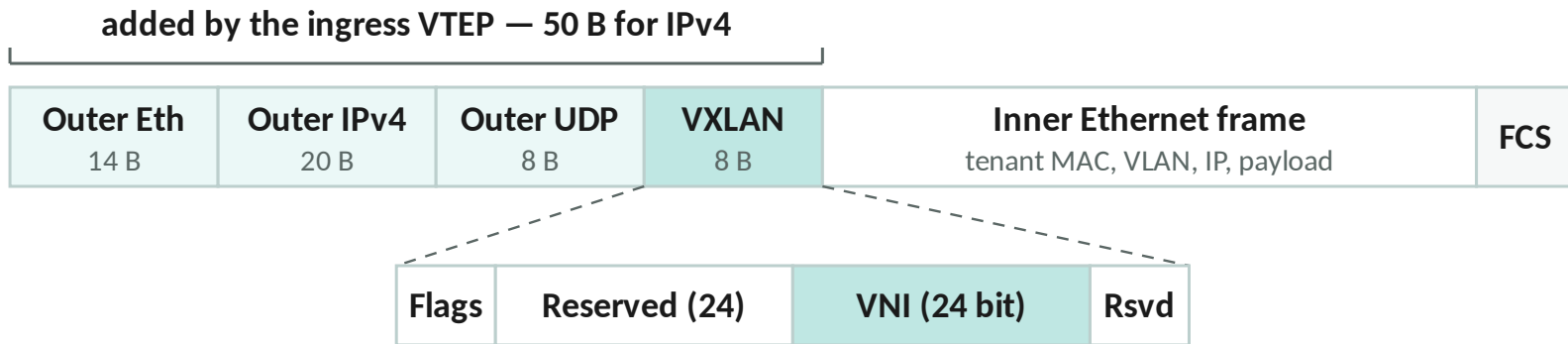
- **millions** of isolated tenant networks, not 4094 VLANs,
- layer-2 adjacency for virtual machines that **move** between racks,
- an **IP underlay** (leaf-spine, ECMP) without spanning tree,
- multi-vendor interoperability at the scale of thousands of switches.

Two answers

OpenFlow: a controller programs every switch (or virtual switch) directly.

VXLAN-EVPN: an overlay data plane plus a standard, distributed BGP control plane.

VXLAN — the overlay data plane



Outer UDP destination port 4789; the source port is a hash of the inner headers, so underlay ECMP spreads tenant flows. A 24-bit VNI gives about 16.7 million segments, against 4094 usable VLAN IDs.

VXLAN (RFC 7348, 2014) tunnels Ethernet frames in UDP/IP between **VXLAN tunnel endpoints (VTEPs)** — leaf switches or hypervisors. The underlay only routes IP; tenant MAC addresses never reach it.

EVPN — a distributed control plane in BGP

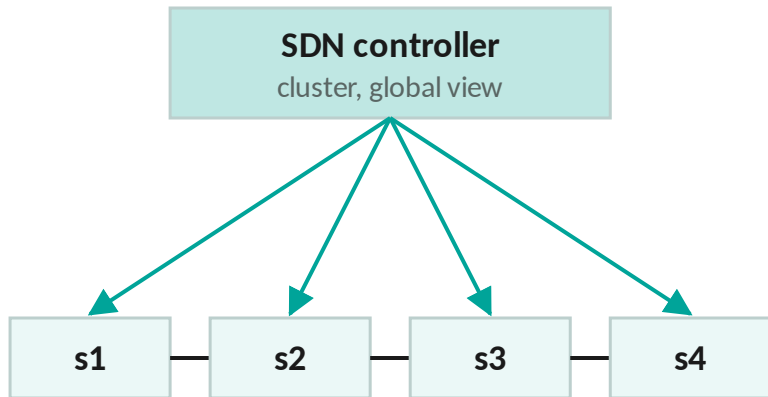
BGP EVPN (RFC 7432, 8365) carries MAC and IP reachability in the MP-BGP address family L2VPN/EVPN. Each VTEP **advertises** its local hosts; the others install the routes. No flood-and-learn in the data plane.

Route type	Carries
1 — Ethernet auto-discovery	multihoming, fast convergence
2 — MAC/IP advertisement	host MAC and IP behind a VTEP
3 — Inclusive multicast	VTEP list for BUM traffic per VNI
4 — Ethernet segment	designated-forwarder election
5 — IP prefix	routed subnets between VNIs (RFC 9136)

Type-2 routes also enable **ARP suppression**: the leaf answers ARP locally from BGP-learned bindings.

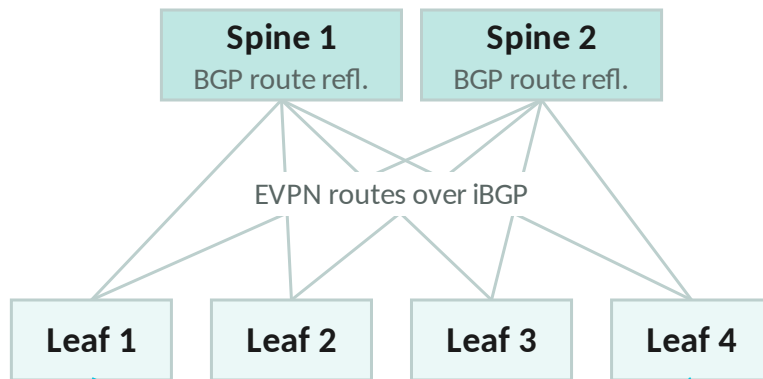
Where the control plane lives

OPENFLOW — CENTRAL CONTROLLER



flow entries pushed into each switch

VXLAN-EVPN — DISTRIBUTED BGP



VXLAN tunnel between VTEPs

OpenFlow: one program computes state for every switch. EVPN: every leaf computes its own state from routes its peers advertise — the control plane is standardized and distributed, as in any IP network.

OpenFlow vs VXLAN-EVPN

Aspect	OpenFlow	VXLAN-EVPN
Control plane	logically centralized controller	distributed: MP-BGP on every VTEP
What is programmed	per-flow match-action entries, any L1-L4 field	per-destination MAC/IP reachability, per VNI and VRF
Data plane	any; OpenFlow mandates no encapsulation	VXLAN over a routed IP underlay
Standardization	ONF; frozen at 1.5.1 (2015)	IETF RFCs, actively evolving
Scaling limit	controller capacity, TCAM size	BGP (Internet-proven), route reflectors
Controller failure	new reactive flows fail (fail-secure)	no controller; BGP reconverges locally
Vendor support	OVS, few hardware switches today	all data-centre switch vendors
Typical use	virtual switches, research, WAN TE	leaf-spine data-centre fabrics, DCI

OpenFlow vs VXLAN-EVPN — the QoS view

OpenFlow

- Per-flow classification, policing (meters), re-marking, and queue selection from one place
- Traffic engineering: explicit path per flow, driven by measured load
- Flow set-up delay for reactive flows; controller is on the critical path of new traffic

VXLAN-EVPN

- QoS is the DiffServ of Lecture 03: the VTEP copies the inner DSCP to the outer IP header so that the underlay can schedule it
- Load spreading by ECMP on the outer UDP source-port hash — statistical, not per-flow placement
- No set-up delay: routes exist before the first packet

Fine-grained control against predictable, distributed behaviour — the same trade-off as IntServ against DiffServ.

Not either/or

OpenFlow driving VXLAN

Network virtualization platforms (OpenStack with OVS/OVN, VMware NSX) use a central controller to program **Open vSwitch** in every hypervisor over OpenFlow — including the VXLAN or Geneve tunnels between hosts.

Controllers managing EVPN fabrics

Fabric controllers and intent-based systems configure the EVPN switches through NETCONF/YANG or gNMI: **centralized management, distributed control.**

The SDN idea — software with a global view defines the network — appears in both. What differs is **which plane is centralized.**

Checkpoint — choose the design

(a) A cloud provider builds a fabric of 2000 racks from switches of two vendors. (b) A university wants to steer individual research flows onto an experimental path and meter them. Which approach suits each, and why?

Predict first · discuss with a neighbour · then reveal

(a) VXLAN-EVPN — standard, multi-vendor, BGP scales to that size, and no controller outage can stop new tenant traffic. **(b) OpenFlow** — per-flow matching on any field, meters, and explicit paths are exactly what it offers, and a campus is the scale the 2008 OpenFlow paper was written for. Either may still be centrally *managed*.

Common mix-ups

SDN vs OpenFlow

SDN is an architecture; OpenFlow is one southbound protocol for it.

Centralized vs single

"Logically centralized" controllers are usually clusters of several servers.

SDN vs NFV

SDN separates control from forwarding; NFV moves middlebox functions into software.

Controller loss vs traffic loss

Installed entries keep forwarding; only traffic needing the controller stops.

VXLAN vs EVPN

VXLAN is the encapsulation (data plane); EVPN is the BGP control plane that fills it.

Meter vs queue

A meter polices (drop/re-mark);
`set_queue` selects a queue whose shaper is configured elsewhere.

Summary

Separation of planes The data plane forwards; the control plane decides. SDN moves the decision into a logically centralized controller.

Architecture Applications — northbound API — controller — southbound API — switches; east-west between controllers.

Flow entries Match fields, priority, counters, instructions, timeouts; the highest-priority match wins, the table-miss entry catches the rest.

Pipeline Goto-Table forward only; the action set executes at the end. Groups for multicast, ECMP, failover; meters for policing.

Channel and messages Switch-initiated TCP/TLS on 6653; controller-to-switch, asynchronous, and symmetric messages.

Reactive set-up Packet-In → Flow-Mod + Packet-Out: the first packet pays the controller round trip.

OpenFlow vs VXLAN-EVPN Central, per-flow, flexible — against distributed, per-destination, BGP-robust.

Beyond OpenFlow P4/P4Runtime for programmable pipelines; SDN as an architecture persists.

Exit questions — use the ideas

1. A switch receives a packet that matches two flow entries with priorities 50 and 200. Which is applied, and what happens if neither exists and the table has no table-miss entry (OpenFlow 1.3)?
2. A controller is 20 ms RTT away from a switch and needs 2 ms per decision. Estimate the extra delay of a new TCP connection's SYN in reactive mode.
3. Why can a VXLAN-EVPN fabric keep accepting new hosts when every management server is down, while a reactive OpenFlow network in fail-secure mode cannot?
4. Which OpenFlow feature implements Lecture 03's token-bucket policer, and which one would you combine it with to prioritize EF-marked voice?

Explain the mechanism, not just its name.

Lab 05 — SDN in Mininet

This lecture described the switch model; Lab 05 lets you **read and write its flow table** by hand:

- Controller-installed entries before and after a ping ↔ reactive set-up (Step 5)
- Empty secure-mode switch, manual `in_port` rules ↔ table miss, fail-secure (Step 5)
- `tc` links with bandwidth, delay, loss ↔ Lecture 03's parameters (Steps 7–8)

```
ssh student@<lab-server>
sudo mn --topo single,2 --mac --controller=none \
    --switch ovsk,protocols=OpenFlow13,failMode=secure
# in the Mininet CLI:
sh ovs-ofctl -O OpenFlow13 dump-flows s1
```

Before you type `add-flow`, predict which packets each entry will match.

References — foundations

- N. McKeown et al., "OpenFlow: Enabling Innovation in Campus Networks," *ACM SIGCOMM CCR*, 38(2), 69–74, 2008.
- N. Feamster, J. Rexford, E. Zegura, "The Road to SDN: An Intellectual History of Programmable Networks," *ACM SIGCOMM CCR*, 44(2), 87–98, 2014.
- D. Kreutz et al., "Software-Defined Networking: A Comprehensive Survey," *Proceedings of the IEEE*, 103(1), 14–76, 2015.
- M. Casado et al., "Ethane: Taking Control of the Enterprise," *ACM SIGCOMM*, 2007.
- N. Gude et al., "NOX: Towards an Operating System for Networks," *ACM SIGCOMM CCR*, 38(3), 2008.
- T. Koponen et al., "Onix: A Distributed Control Platform for Large-scale Production Networks," *USENIX OSDI*, 2010.
- S. Jain et al., "B4: Experience with a Globally-Deployed Software Defined WAN," *ACM SIGCOMM*, 2013.
- P. Bosshart et al., "P4: Programming Protocol-Independent Packet Processors," *ACM SIGCOMM CCR*, 44(3), 2014.
- L. Peterson et al., *Software-Defined Networks: A Systems Approach*. Systems Approach LLC, 2021 (online, sdn.systemsapproach.org).
- J. F. Kurose, K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed., ch. 4.4 and 5.5. Pearson, 2021.

References — standards

- Open Networking Foundation, *OpenFlow Switch Specification*, version 1.3.5 (TS-023) and 1.5.1 (TS-025), 2015.
- E. Haleplidis et al., "Software-Defined Networking (SDN): Layers and Architecture Terminology," RFC 7426, 2015.
- L. Yang et al., "Forwarding and Control Element Separation (ForCES) Framework," RFC 3746, 2004.
- M. Mahalingam et al., "Virtual eXtensible Local Area Network (VXLAN)," RFC 7348, 2014.
- A. Sajassi et al., "BGP MPLS-Based Ethernet VPN," RFC 7432, 2015.
- A. Sajassi et al., "A Network Virtualization Overlay Solution Using Ethernet VPN (EVPN)," RFC 8365, 2018.
- J. Rabadan et al., "IP Prefix Advertisement in Ethernet VPN (EVPN)," RFC 9136, 2021.
- ETSI GS NFV 002, *Network Functions Virtualisation (NFV); Architectural Framework*, 2013.

Thank you for your attention

Jan Rozhon

+420 596 995 900 · jan.rozhon@vsb.cz

www.vsb.cz